RESULTS REPORT · DEEP REINFORCEMENT LEARNING · SPRING 2026

Learning when to think: does the model learn to spend less on easy and more on hard?

An RL policy with three actions (continue, refine, terminate) trained on Qwen2.5-Math-7B-Instruct with LoRA, GRPO, ALP reward, and DeGRPO weighting. Tested on MATH-500.

Ivan Andhika · u1590903 | Hao Ren · u1527543 | Ammon Gleason · u1221580

University of Utah · CS 6955 Final proposal (PDF) Raw result · docs/results_h1_h3.md

01 The problem

Large language models think for the same amount of time on every question. They use the same number of tokens for "What is 2 + 2?" and for a hard olympiad problem.

This is a waste. Easy questions get too many tokens. Hard questions sometimes stop too early and get the wrong answer. We want a model that **chooses how much to think** based on the question.

> **Goal in one line.** Train a small policy on top of an LLM so it can decide: keep thinking, fix a mistake, or stop and answer.

02 The idea: three actions

We treat reasoning as a small game (an MDP). At every step the model picks one of three actions:

CONTINUE	REFINE	TERMINATE

Keep writing the next reasoning step. Use this when the answer is not ready yet.

Stop, look back, and fix the last steps. Use this when something feels wrong.

Stop reasoning and write the final answer. Use this when the answer is clear.

The model emits a small *control token* for the action, then writes the matching text. This makes the action choice trainable with normal RL.

03 Reward: ALP (Adaptive Length Penalty)

The reward gives a point for a correct answer and takes points away for a long answer. The penalty is bigger when the question is easy.

$$R_k = r_{\text{acc},k} - \beta \cdot \max\big(0,\, \text{SR}(q)\big) \cdot \frac{n_{\text{tokens},k}}{L_{\max}}$$

$r_{\text{acc},k}$: 1 if the answer is correct, else 0. **SR(q)**: how often the group solves question q (a proxy for "easy"). $n_{\text{tokens},k}$: how long the answer is. **β, $L_{\max}$**: fixed scale factors.

In plain words: **easy + long = big penalty**. **hard + long = small penalty**. So the model learns to spend tokens where they help.

04 Training: GRPO + DeGRPO + LoRA

We use **GRPO** (Group Relative Policy Optimization). For each question we sample a small group of answers and use the group as a baseline. No separate critic model needed.

DeGRPO is our small change. The control tokens (action choices) get a higher gradient weight than the normal text tokens. This stops the action signal from being washed out by the long text.

$$\mathcal{L} = -A_k\big(w_{\text{ctrl}}\, \log p_{\text{ctrl}} + w_{\text{resp}}\, \log p_{\text{resp}}\big) + \lambda_{\text{KL}} \cdot \text{KL}$$

Default weights: w_{ctrl} = 2.0, w_{resp} = 1.0. KL term keeps the policy close to the base model.

BASE MODEL

ADAPTER

Qwen2.5-Math-7B-Instruct

LoRA, `r=16`, `α=32` on attention + MLP

ALGORITHM

GRPO + ALP reward + DeGRPO weighting

TRAIN DATA

MATH-500 train subset, `n=32`, 2 epochs

Important note for H3. The training config used `allow_refine: false`, so the policy only really learned `continue` and `terminate`. The `refine` action can still appear at decode time, but the model was not trained to use it well.

05 Experimental setup

We test on **MATH-500**, a hard high-school competition math benchmark with 5 difficulty levels (1 = easiest, 5 = hardest) and several subjects.

- **Eval size:** `n = 100` per checkpoint (CHPC jobs are capped at 2 hours).
- **Resume:** per-problem JSONL, so a killed job restarts from the last solved item.
- **Methods compared:** CoT baseline, Direct (no reasoning), Adaptive DeGRPO with and without `refine`, Adaptive Vanilla GRPO with and without `refine`.
- **Pipeline:** see docs/results_h1_h3.md for full commands.

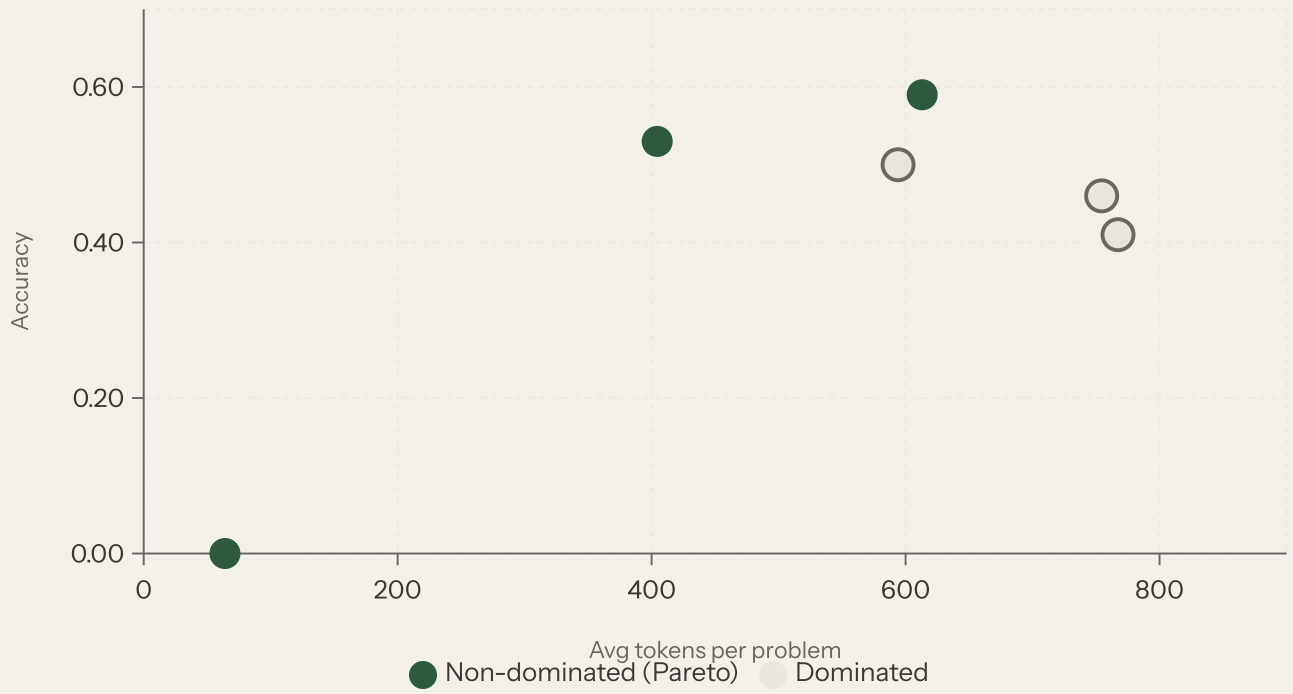
Statistical caveat. At `n=100` the 95% CI near accuracy 0.5 is about ±10 points. So differences smaller than that are not statistically clear.

06 H1 — Better accuracy/efficiency trade-off

Hypothesis. The adaptive policy with ALP + DeGRPO should give a better Pareto frontier (more accuracy per token) than CoT and vanilla GRPO.

Accuracy vs avg tokens per problem (MATH-500, n=100)

Top-left is better (high accuracy, fewer tokens). Green = non-dominated (Pareto). Grey ring = dominated.



METHOD	CORRECT / TOTAL	ACCURACY (95% CI)	AVG TOKENS	COST / CORRECT	PARETO
CoT baseline	53 / 100	0.530 [0.433, 0.625]	404.4	763.0	non-dominated
Direct baseline	0 / 100	0.000 [0.000, 0.037]	64.0	6400.0	non-dominated
Adaptive DeGRPO (+refine)	46 / 100	0.460 [0.366, 0.557]	754.4	1640.1	dominated
Adaptive DeGRPO (-refine)	59 / 100	0.590 [0.492, 0.681]	613.1	1039.2	non-dominated
Adaptive Vanilla (+refine)	41 / 100	0.410 [0.319, 0.508]	767.3	1871.4	dominated
Adaptive Vanilla (-refine)	50 / 100	0.500 [0.404, 0.596]	594.1	1188.3	dominated

PARTIALLY SUPPORTED

The best adaptive run is DeGRPO without refine (0.59 accuracy, 613 tokens). It is the only adaptive point on the Pareto frontier and beats vanilla GRPO without refine (0.50).

But its CI overlaps with CoT (0.53), so the gain over CoT is not statistically clean at `n=100`. All `+refine` rows are dominated.

07 H2 — More compute on harder problems

Hypothesis. A learned policy should give more tokens (and more steps) to harder problems.

`+refine: Spearman(level, tokens) = 0.360`

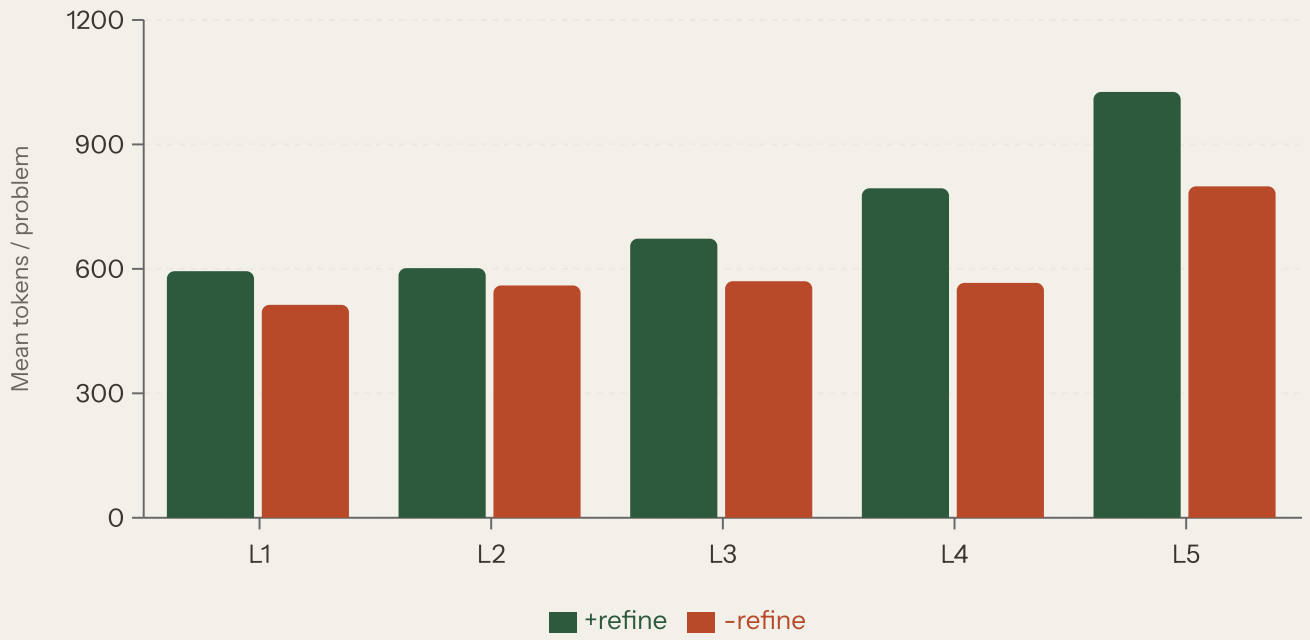
`+refine: Spearman(level, steps) = 0.138`

`-refine: Spearman(level, tokens) = 0.288`

`-refine: Spearman(level, steps) = 0.149`

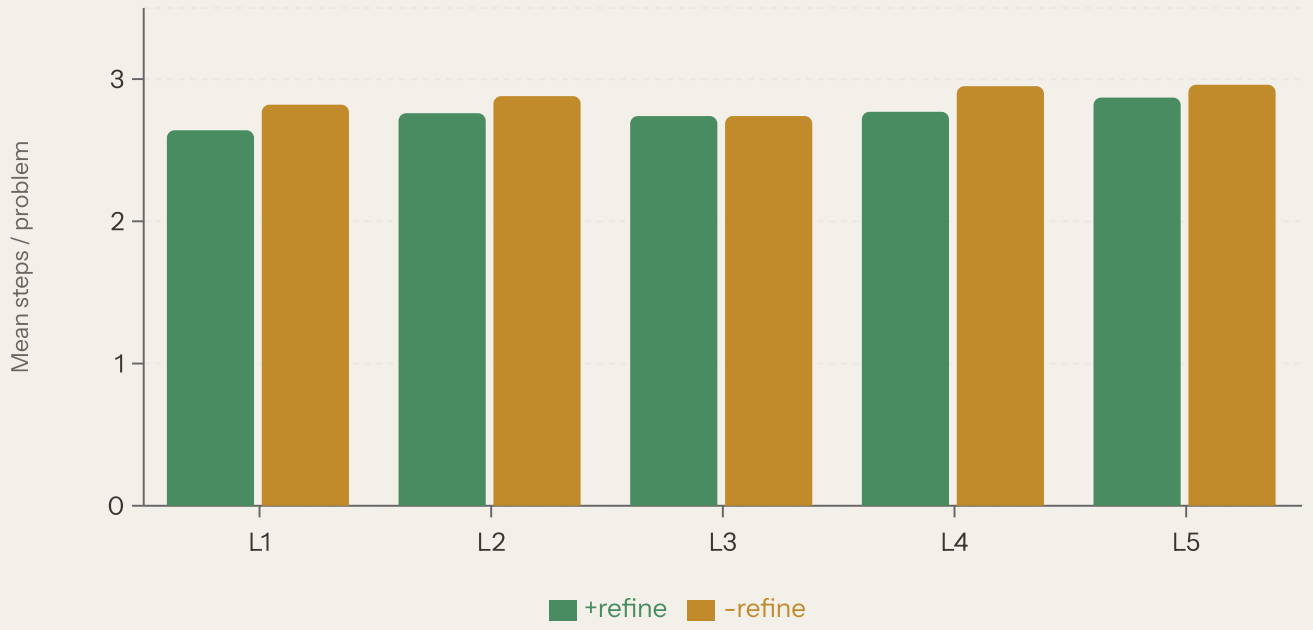
Mean tokens per problem by difficulty level

Tokens tend to rise from L1 (easy) to L5 (hard) for both decode modes.



Mean steps per problem by difficulty level

Step count stays almost flat (~2.6–3.0). The model uses longer text inside a step, not more steps.



Per-level numbers (Adaptive DeGRPO)

LEVEL	N	+REFINE: TOKENS	+REFINE: STEPS	+REFINE: ACC	-REFINE: TOKENS	-REFINE: STEPS	-REFINE: ACC
1	11	594.4	2.64	0.545	513.3	2.82	0.909
2	25	601.7	2.76	0.720	560.0	2.88	0.760
3	19	672.8	2.74	0.474	570.2	2.74	0.789
4	22	794.2	2.77	0.273	566.2	2.95	0.500
5	23	1026.5	2.87	0.304	798.9	2.96	0.174

WEAKLY SUPPORTED

Yes for tokens, no for steps. Mean tokens rise clearly with difficulty (594 → 1027 for +refine). But step count is almost flat. So the model spends more *inside* a step on hard problems, not more steps. Spearman is positive but only moderate (0.29–0.36).

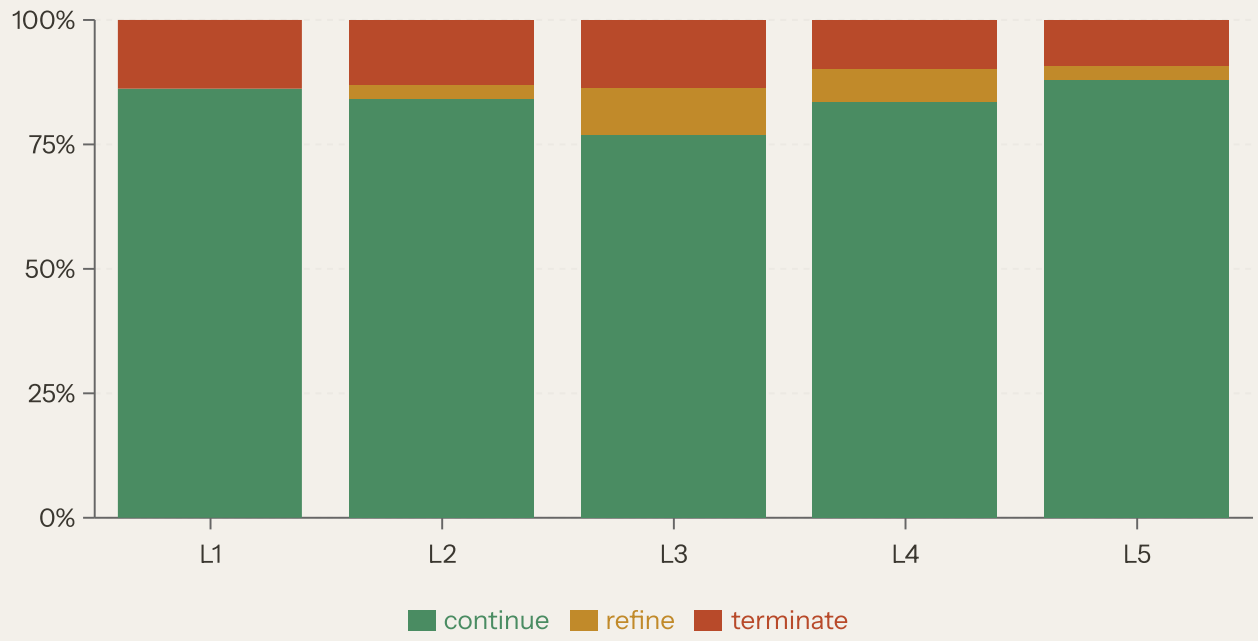
08 H3 — Action mix changes with difficulty

Hypothesis. Easy problems should get more `terminate`. Hard problems should get more `continue` and `refine`.

Note: the policy was trained with `allow_refine: false`. So the cleanest test is the `+refine` decode (refine token allowed at test time) and the `-refine` decode (refine forced off).

Action share by level — +refine decode

Each bar adds up to 100%. Lower terminate share on harder levels would support H3.



LEVEL	N	FRAC CONTINUE	FRAC REFINE	FRAC TERMINATE
1	11	0.862	0.000	0.138
2	25	0.841	0.029	0.130
3	19	0.769	0.096	0.135
4	22	0.836	0.066	0.098
5	23	0.879	0.030	0.091

Action share by level — -refine decode (refine forced off)

Two-action regime: continue vs terminate. Refine is always 0 here.

LEVEL	N	FRAC CONTINUE	FRAC REFINE	FRAC TERMINATE
1	11	0.839	0.000	0.161
2	25	0.903	0.000	0.097
3	19	0.885	0.000	0.115
4	22	0.985	0.000	0.015
5	23	0.912	0.000	0.088

PARTIALLY SUPPORTED

Terminate share is higher on level 1 than on level 5 in both decodes (0.138 vs 0.091 with refine; 0.161 vs 0.088 without). That matches H3 in spirit. But the trend is noisy (level 4 is an outlier with very few terminates) and refine is rarely picked because the model was never trained to use it. A clean 3-action H3 needs a retrain with `allow_refine: true`.

09 What worked, what did not

H1 · PARETO

Partial

H2 · ADAPTIVE COMPUTE

Weak

H3 · ACTION MIX

Partial

DeGRPO -refine is non-dominated. CIs vs CoT overlap.

Tokens grow with level. Steps are flat.

More terminate on easy. Refine never really learned.

What worked. DeGRPO without refine is the best adaptive setting. It beats vanilla GRPO without refine at almost the same token cost (0.59 vs 0.50 accuracy, ~613 vs ~594 tokens). The model also spends more tokens on harder levels, which is the main behavior we wanted.

What did not work. The `refine` action did not help. All `+refine` runs are dominated. The main reason is simple: training had `allow_refine: false`, so the model was never rewarded for using refine well. Step count also stays flat, so "thinking longer" here means "longer single steps", not "more steps".

Headline answer. The pipeline does push the policy in the right direction (more tokens on hard, more terminate on easy, DeGRPO > vanilla). But the size of the win is small and not statistically clean at `n=100`. We see a *signal*, not a *proof*.

10 Limitations and next steps

Limits in this report

- **Train set is tiny (n=32):** training accuracy went 0.59 → 0.49 between epochs, which hints at instability.
- **Eval set is small (n=100):** 95% CI near accuracy 0.5 is about ±10 points. Most differences are inside this gap.
- **Reduced action space at train time:** `allow_refine: false`, so H3 is really a 2-action test (continue vs terminate).
- **Vanilla GRPO uses the same small data,** so the H1 ablation inherits the same noise.
- **No external PRM, no AIME, no full MATH, no Pass@k baseline.** All out of scope for this proposal.

Next steps if we had more time

- Retrain with `allow_refine: true` and a real refine reward signal so H3 is a true 3-action test.
- Scale train data from `n=32` to several hundred MATH problems and run more epochs.
- Push eval to `n=500` (full MATH-500) so CIs shrink to roughly ±4 points.
- Add self-consistency Pass@k as a stronger compute baseline for H1.

Source data: docs/results_h1_h3.md. Code: analyze_h1_pareto.py, analyze_h2_h3.py. CHPC eval logs under `chpc/results/eval/`.